

AutoRemediate AI

Autonomous Vulnerability Remediation Plan & Incident Response Hub

Project 7 · Capstone Project Report

Digital Egypt Pioneers Initiative (DEPI)

Cyber Security Incident Response Analyst Track

Evaluator: **Eng. Ahmed Attia**

DEPI Team 3

Marc Wael (Team Lead & Engineering) · Mohey Elden Mostafa · Jerome Arsany

Noureen Elsayed · Hassan Ibrahim · Philopater Helmy

github.com/nourinsayed55-ui/autoremediate-ai

AutoRemediate AI — Project Report

1. Executive Summary

AutoRemediate AI is an autonomous, human-governed **SOAR-lite (Security Orchestration, Automation and Response)** platform built for the DEPI Cyber Security Incident Response Analyst capstone. The system conducts a live vulnerability assessment against a deliberately vulnerable target, ranks the discovered flaws using a transparent scoring model, recommends the most appropriate remediation for each, and — only after explicit human approval — executes that remediation in real time over an encrypted SSH channel, verifying the result two independent ways and logging every action to a cloud database.

Against a live scan of the target, the engine ingested **43 real findings (9 Critical, 6 High, 20 Medium, 8 Low)**, prioritised the 15 Critical/High items, and demonstrably closed real network services on the target with a measured time-to-remediate well under the 60-second target. Every change required a human decision, satisfying the project's central safety requirement.

2. Problem Statement & Objective

In a traditional workflow, scanners identify an exposed service, a report is written, and a technician applies a fix days later — leaving a window in which the vulnerability is exploitable. The mean-time-to-remediate is measured in days, while an attacker needs only minutes.

Primary objective: conduct a comprehensive vulnerability assessment on an active target, isolate the identified flaws by severity, and execute a structured, programmatic, human-approved remediation plan that systematically hardens the target — reducing remediation time from days to seconds while never removing the human from the decision.

3. Assessment Scope & Lab Environment

All activity is confined to an isolated virtual lab. The real internet is explicitly out of scope; the target is never exposed externally.

Component	Role in the Project
Windows 11 host	Runs the Python engine, Claude Code, and the web console.
Kali Linux VM	Hosts the Nessus scanner; reachable over the lab network.
Metasploitable2 VM	Deliberately vulnerable Ubuntu 8.04 target (the system under test).
Nessus Essentials	Industry-standard vulnerability scanner; REST API with X-ApiKeys auth.
Python + Paramiko	Orchestration engine and encrypted SSH remediation executor.
Supabase (PostgreSQL)	Cloud relational store for findings and remediation logs.
Flask Web Console	Browser interface where the operator approves each remediation.
Claude Code (assistant)	AI pair-programmer used to build, test, and document the system.

4. System Architecture

The platform follows a decoupled, four-layer architecture, so each responsibility can be developed and reasoned about independently:

- **Detection Layer** — Nessus audits the target over the host-only network and produces structured findings.
- **Orchestration Layer** — a class-based Python engine ingests the findings, computes a priority score, and selects a recommended remediation.
- **Remediation Transport Layer** — Paramiko opens an encrypted SSH session and runs the approved command on the target.
- **Presentation Layer** — a Flask web console plus Supabase surface live counters, the activity log, and verification status.

Data Flow

Target → Nessus scan → JSON via REST API → Python engine → human approval → SSH remediation → two-way verification → Supabase & dashboard

5. Core Design Pillars (Evaluator-Tested)

Three behaviours were engineered into the system because the evaluator verifies each of them by hand.

5.1 Mandatory Human-in-the-Loop Approval

For every Critical/High finding the engine prints a decision card and blocks until the operator chooses. The configuration flag **require_manual_approval = true** is the default and has no silent bypass: no code path executes a change to the target without an explicit keystroke or button click. Every decision — approve, alternative, or skip — is written to the audit log with the operator's choice, proving a human authorised each action.

5.2 Prioritised, Preferred Recommendations

Findings are ranked by a deterministic **Priority Score** computed from real Nessus fields, and each finding's recommended action is selected by walking a **Remediation Preference Ladder** — the best, safest, most feasible fix is offered first, with alternatives and containment options below it.

5.3 Real, Verified Execution

On approval, the system runs the real command on the live target, then verifies success two independent ways before declaring it: an on-target service/port check and an external socket probe. Only if both agree is the action logged as Success.

6. Prioritisation Criteria

The Priority Score determines the order in which findings are reviewed. All inputs are taken from the real scan — nothing is fabricated.

Factor	Points
--------	--------

Severity: Critical / High / Medium / Low	100 / 70 / 40 / 10
Known backdoor / public Metasploit module / actively exploited	+40
Public proof-of-concept, not weaponised	+20
No authentication required to exploit	+15
Service confirmed listening and reachable	+15

Findings are sorted highest-first; ties are broken by lower port number, then by newer CVE.

7. Remediation Preference Ladder

For each finding the engine recommends the first rung below that is both safe and feasible; lower rungs become the offered alternatives.

- **1. Patch / Upgrade** — the root-cause fix. Usually infeasible on the end-of-life Ubuntu 8.04 target, which the engine states honestly.
- **2. Stop / Disable the service** — preferred for backdoored or abandoned services; removes the attack surface.
- **3. Harden / Reconfigure** — preferred when the service must stay up (for example, SSH: disable root login, then restart).
- **4. Network containment** — firewall (iptables) DROP on the port; the standing alternative on every finding.
- **5. Monitor only** — log and alert, no change; the explicit safe fallback.

Hard safety rule: the engine never stops or firewall-blocks port 22, because that would sever its own SSH control channel. For SSH it may only harden. This blast-radius reasoning is enforced in code, not merely documented.

8. Relational Data Model

Two normalised PostgreSQL tables (hosted on Supabase) record findings and the actions taken against them. The engine creates them itself via the connection string; every remediation writes a real, queryable row.

Table A — Vulnerabilities

Column	Type
vuln_id	SERIAL PRIMARY KEY
cve_reference	TEXT
plugin_name	TEXT
severity_level	TEXT CHECK (Critical/High/Medium/Low)
target_port	INTEGER

Table B — Remediation_Activity_Logs

Column	Type
log_id	SERIAL PRIMARY KEY
vuln_id	INTEGER REFERENCES Vulnerabilities

timestamp	TIMESTAMPTZ DEFAULT now()
command_dispatched	TEXT
operator_decision	TEXT
execution_status	TEXT CHECK (Success/Failed/Retrying/Skipped)

9. Implementation & Engineering Depth

Building a system that performs real actions on a 2008-era target surfaced genuine engineering problems. Solving them is what distinguishes a working system from a simulated one.

9.1 Legacy SSH Handshake

Metasploitable2's OpenSSH 4.7 offers only the legacy ssh-rsa / SHA-1 host-key algorithms, which modern Paramiko rejects by default. The remediator was patched to re-enable those legacy algorithms for this specific target so the encrypted session can be established.

9.2 sudo-over-SSH Hang

Privileged commands invoked through a non-interactive SSH channel caused **sudo** to block forever waiting for a password. This was resolved with **sudo -S**, feeding the password via stdin, combined with hard command timeouts so no remediation can ever hang the run.

9.3 Pre-systemd Service Management

Ubuntu 8.04 predates systemd, so **systemctl** fails silently. All remediation commands use the Upstart-safe **/etc/init.d/<service>** form, with init.d script names verified against the live host (for example samba, postgresql-8.3).

9.4 Per-Action Verification

Each action type is verified by the correct method: stop/containment by confirming the port is unreachable, hardening by confirming the configuration change took effect, and monitor-only is always treated as handled — preventing false failures.

10. Results

A live Nessus Basic Network Scan (Scan ID 12) of the target produced the following:

Metric	Result
Total findings ingested (live scan)	43
Critical / High / Medium / Low	9 / 6 / 20 / 8
Critical + High prioritised for action	15
Top-ranked findings (score 155)	Bind Shell (1524), UnrealIRCd backdoor (6667)
Time-to-remediate per finding	Under 60 seconds (target met)
Human approval on real changes	100% — enforced, no auto-approval

Live Verification Example

Before: Test-NetConnection 6667 → TcpTestSucceeded : True

After: Test-NetConnection 6667 → TcpTestSucceeded : False

An approved remediation closed a real port on the target, confirmed from an independent tool — the core proof the system is genuine.

11. Key Performance Indicators

- **Mean-Time-To-Remediate (MTTR):** under 60 seconds per finding, versus days for manual remediation.
- **Remediation success rate:** 100% execution success on approved critical findings during testing.
- **System stability:** 0% unintended service disruption — verification runs before a finding is marked resolved, and the engine never severs its own control channel.

12. Deliverables & Documentation

All required graduation artifacts are produced as real, formatted documents and committed to the project repository with a clean, week-by-week history.

Deliverable	Location in Repository
Tool Configuration Documentation	docs/Tool_Configuration_Documentation.docx
Assessment Scope Document	docs/Assessment_Scope_Document.docx
Vulnerability Scan Report	docs/Vulnerability_Scan_Report.docx
Initial Analysis Document	docs/Initial_Analysis_Document.docx
Prioritization Report	docs/Prioritization_Report.docx
Remediation Plan	docs/Remediation_Plan.docx
Verification Report	docs/Verification_Report.docx
Final Remediation Report	docs/Final_Remediation_Report.docx
System Analysis & Design (UML/DFD)	docs/System_Analysis_Design.md
Relational data model	sql/schema.sql (live in Supabase)

The repository additionally contains the engine source (backend/), unit tests (tests/, 53+ passing), a local web console, a README, and PROJECT_CONTEXT.md. Live credentials are never committed: config.json is git-ignored and a placeholder config.example.json is committed instead.

13. Conclusion

AutoRemediate AI delivers a working, defensible answer to the project brief: a prioritised, human-approved, and independently verified autonomous remediation system that takes real action on a live target in seconds rather than days. It satisfies the evaluator's three hand-tested requirements — no auto-approval, prioritised recommendations with alternatives, and genuine real-time remediation — and ships every required deliverable in a cleanly documented repository.